Residue Number System Quantization Limitations

Jofeth Abello

May 6, 2026

1 Introduction

Quantization in Deep Learning have been used to optimize the distributed training and inference of neural network models. It is one of the gradient compression methods used for optimizing machine learning pipeline from training to inference. It generally finds application in communication by reducing the volume of data that needs to be transferred in a bandwidth limited interconnection, and in computation by reducing the number of bits that need to be operated on with multiplication or addition. The idea is to reduce the bit width of gradients, typically in FP32/16/8, or in integers such as INT8. However, it presents two inherent mathematical problems: it reduces the precision supported, and it is a non-injective mapping because we are mapping from a continuous domain into a discrete domain. The former, comes the fact that we are using less number of bits. The latter, follows from Cantor's diagonal argument, which proves that the set $\mathbb{R}$ is uncountably infinite, while $\mathbb{N}$ is countably infinite. Thus, they are in different orders of infinite, and no bijective mapping (one-to-one) between them can exist.

2 Assumptions

This report distinguishes two layers of representing a number. The logical layer that is the format in which we see the number (e.g. 3.14), referred as number or number with fraction. And the physical layer that is the format in which they are stored and represented in hardware in bits (e.g. IEEE 754 single-precision), referred as FPX or floating-point. When talking about floating-points we are referring to the standard by IEEE 754.

3 Hypothesis

To solve both problems, this report proposes to do a fixed-point logical layer of a a number with fraction. We treat a number as of the form A.B, where A is the

integer part and B is the fractional part. For example, 3.14 is represented as A = 3 and B = 14. To reduce the bit width from FP32 into a logical representation that uses less than 32 bits, Residue Number System (RNS) is proposed. To do so, we use a set of moduli S to convert A and B into RNS integers. Hence, our constraint is representing FP32 values with RNS quantization using number of bits strictly lower than 32.

4 Reasoning

In this section, we will show that there are properties of RNS that do not allow us to reduce the number of bits and fully represent the precision provided by FP32. The problem stems from the range of representable values of FP32. The maximum and minimum of FP32 according to IEEE 754 standard is $\pm 3.4028235 \times 10^{38}$, derived from $(2 - 2^{-23}) \times 2^{127}$. Thus, to represent the maximum and minimum values of FP32, approximately 39 decimal digits are needed. Moreover, the minimum positive subnormal value is $2^{-149} \approx 1.4 \times 10^{-45}$, which needs approximately 46 digits. Hence, a set of moduli $S = \{m_1, m_2, \ldots, m_n\}$ needs to have a dynamic range $M = \prod_{i=1}^{n} m_i \geq 10^{46}$.

4.1 Problem I: RNS length $<$ 10 digits

The RNS representation $Z = (N \mod m_1, N \mod m_2, \ldots, N \mod m_n) = (r_1, r_2, \ldots, r_n)$ must have a length strictly less than 10 digits. Such constraint, is due to the goal of using less than 32 bits. In fact, using 32 bits, it is possible to represent an integer with a value of at most $2^{32} - 1$, which has a length of at most 10 digits. Hence, in order to use fewer bits, the RNS length is limited to at most 9. To do so, two approaches are possible, either use few large moduli or many small moduli. These choices leads to a further problem.

4.2 Problem II: Modular Arithmetic

In modular arithmetic, integers can be represented as $N = mq + r$, where m is the modulo, q is the quotient, and r is the residue (or remainder). A consequence of this is that $r \leq m$. Hence, bits required to represent N as its residue is upper bounded by the bits required by the modulo m and the following lemmas are true:

(i) Big modulo M produces big residue, use few high number of bits, $r \leq M$

(ii) Small modulo m produces small residue, use many low number of bits, $r \leq m$

4.2.1 Lemma: Big Modulo means Few High Number of Bits

Proof. The problem consists choosing a set-moduli $S = \{m_1, m_2, \ldots, m_n\}$ s.t. the length of $Z = (N \mod m_1, N \mod m_2, \ldots, N \mod m_n) = (r_1, r_2, \ldots, r_n)$

residues is less than 10 digits while reaching a dynamic range $M \geq 10^{46}$. Thereby, we need to find n pairwise coprime numbers that has such product and ensure that at most 9 positions are used, in accordance to problem 4.1.

To restrict the problem of finding pairwise coprime numbers, a series of problem reductions are gonna be done to the original problem.

4.2.1.1 Reduction to Stars and Bars

It is convenient to solve the problem of assigning digits in at most 9 positions (lets call them stars, represented as *********), which is equivalent to finding the possible groupings of 9 identical objects by assigning at most 9 separators | s.t. it has the desired dynamic range. An example is the following.

$$*|*|*|*|*|*|*|*|*|$$

where each * is in radix-10 (0, 1, ..., 9). Observe that there is no need to exhaustively search every possible factorization, it is sufficient to prove that with factors that use in total 9 digits, it is possible to cover the range $[0, 10^{46}]$.

4.2.1.2 Reduction to Max Product of Integers with Constrained Factors Length K

The problem can be transformed into finding the max product that can be obtained using factors where the sum of their lengths is exactly $K = 9$.

$$\begin{aligned} \max \quad & \prod_{i=1}^{n} a_i \\ \text{s.t.} \quad & \sum_{i=1}^{n} \text{digits}(a_i) = K \\ & a_i \in \mathbb{N} \end{aligned} \tag{1}$$

To achieve this, remember that the Positional Number System (PNS) represents a number $N = \sum_{i=1}^{n} b^{i-1}$, where n is the number of digits and b is the radix (or base). Based on this system, the higher number of digits, the bigger the weights of each position. Moreover, to maximize the product it is necessary to use the largest digit, which is 9. Given the use of the same digit, the problem transforms into maximizing the weights. In radix-10 the weights are powers of 10. So this is equivalent to maximizing the sum of their exponent.

Observe that each possible factor a_i has a form of finite repeating 9s (i.e. 99...9), which can be expressed in scientific notation as $a_i = 0.99...9 * 10^k$, where k is the number of 9s.

Because $0.9999... = 1$ and the factors are terminating 9s. It follows that $a_i = (0.99...9 * 10^k) \leq 10^k$. The problem has now been reduced into finding factors of the form $a_i \approx 10^k$, which maximizes the product.

$$\max \quad \prod_{i=1}^{n} 10^{w_i} = 10^{\sum w_i} = 10^K \tag{2}$$

Given that k is the number of 9s of each factor which is equals the length of each factor, moreover the total length of factors must be 9. Thus, the maximum product we can obtain is at most $10^K = 10^9$.

Example: $99,999 * 9,999$ is equivalent to $(0.99999 * 10^5) * (0.9999 * 10^4)$. Because $0.99999... = 1$ and we have terminating 9s, the product would be at most $10^5 * 10^4 = 10^9$.

However, $10^9 << 10^{46}$. Since, we cannot cover the range $[0, 10^{46}]$ using maximum factors, then product of pair-wise coprime factors cannot also cover the same range. ■

4.2.2 Lemma: Small Modulo means Many Low Number of Bits

Proof. Due to problem 4.1, using small moduli means using exactly 9 pairwise coprime single digit numbers. However, a similar argument could be made as case (i), where we determine whether the set moduli S could cover the range. Setting each modulo to 9, we could cover at most 9^9. Hence, it is insufficient. ■

5 Conclusion

Using RNS to cover the whole range of IEEE 754 in single-precision using a fixed-point logical representation of the number with fraction, while using less than 32 bits is unfeasible due to said constraint and the property of modular arithmetic where residues are upper bounded by the modulo. Hence, the total length of the chosen moduli constraints the range of possible unique representations of the dynamic range, and consequently constraints the number of bits required. Thus, the problem of not being able to find a bijective function between $\mathbb{R}$ and $\mathbb{N}$ still persists, because we are unable to find an injective mapping from floating-points to integers. The insight we can gain from this report is that the set of moduli determines the number of bits required, as such saving on the number of bits means using a set of moduli with low dynamic range. However, this limitation applies in a single-channel communication. We see multiple potential research directions in the use of RNS to provide quantization and parallelism: multiple-channel communication scenarios where we send each residue in a different channel. Error correction by providing multiple copies of the same residue. Communication and computations (e.g. model inference) where the primary data types operated are integers or real numbers with limited precision. Hence, exploring RNS in communication and computation optimization are still valuable research directions to exploit its reduced latency for arithmetic operations and residue independence which allows data parallelism.